

学号 20201008303 姓名 王韵菲

(3) 选择【Edit】->【Perferences...】，在 Font 选项设置字体是 Fixedsys，Script 是 CHINESE GB2312。

(4) 选择【Edit】->【DebugRel Settings...】，在 DebugRel Settings 对话框的左边选择 ARM Linker 项，设置链接地址。连接器选项：RO Base 文本框中填入 0x40000000。地址 0x40000000 是开发板上 SDRAM 的真实地址，是由系统的硬件决定的；RW Base 文本框中填入 0x40003000 指的是系统可读写内存的地址。也就是说，在 0x40000000—0x40003000 之间是只读区域，存放程序的代码段，从 0x40003000 开始是程序的数据段。Layout 选项卡：它用来安排一些输入段在映像文件中的位置。Place at beginning of image 选项组用于指定将某个输入段放置在它所在的运行时域的开头。包含复位异常中断处理程序的输入段通常放置在运行时域的开头。这里，在 Object/Symbol 文本框中指定目标文件的名称 init.o，在 Section 文本框中指定输入段的名称 init，从而确定了 init.s 源文件中的 init 输入段位指定的输入段。

(5) 选择【Project】->【Debug】，或者按下快捷键 F5。IDE 环境就会启动 AXD 调试软件，接着可以执行单步、全速运行调试。注意：本实验使用软件仿真，所以要在 AXD 中选择【Options】->【Configure Target...】菜单，然后在 Choose Target 窗口中选用 ARMUL 软件仿真。

实验结果图：

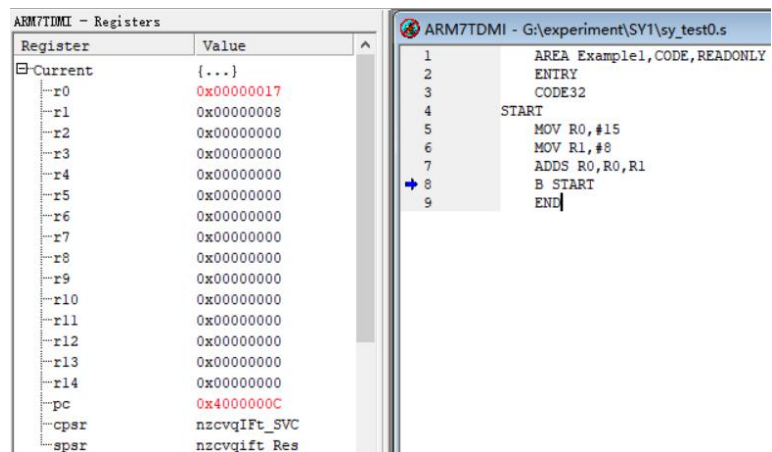


图 1 查看寄存器图

2、实验练习 1

(1) 输入以下程序，并查看程序执行过程中内存地址 0x1000 单元处的数据变化，并记录其变化值。给；后的语句加上注释

(2) 添加注释：

```

MI - G:\experiment\SY1\sy_test1.s
x equ 45 ;x=45
y equ 64 ;y=64
stack_top equ 0x1000 ;stack_top起始地址为0x1000
area code1, code, readonly
entry
code32
_start ;程序开始的标号
mov sp, #stack_top
mov r0, #x ;r0=45
str r0, [sp] ;[sp]=45, [0x1000]=45
mov r0, #y ;r0=64
ldr r1, [sp] ;r1=45
add r0, r0, r1
str r0, [sp] ;[sp]=r0, [0x1000]=109
stop
| B stop ;跳转到stop标签，则一直停留在此
end

```

图 2 注释图

(3) 单步运行了 3 步后的寄存器 (r0=0x0000002D, sp=0x0000800C) 及存储器首地址值 (45)

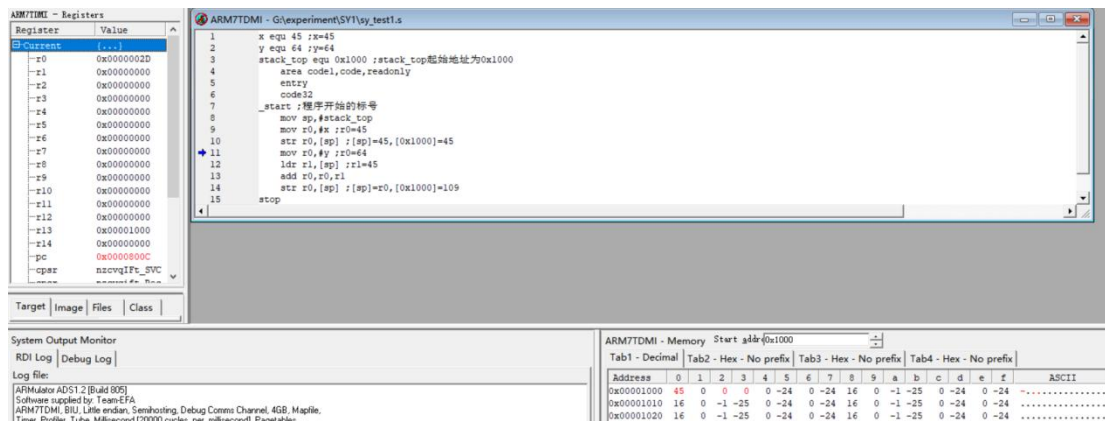


图 3 运行三步后的寄存器及存储器值

(4) 运行结束的寄存器 (r0=0x0000006D, r1=0x0000002D, sp=0x0000801C) 及存储器首地址值 (109)

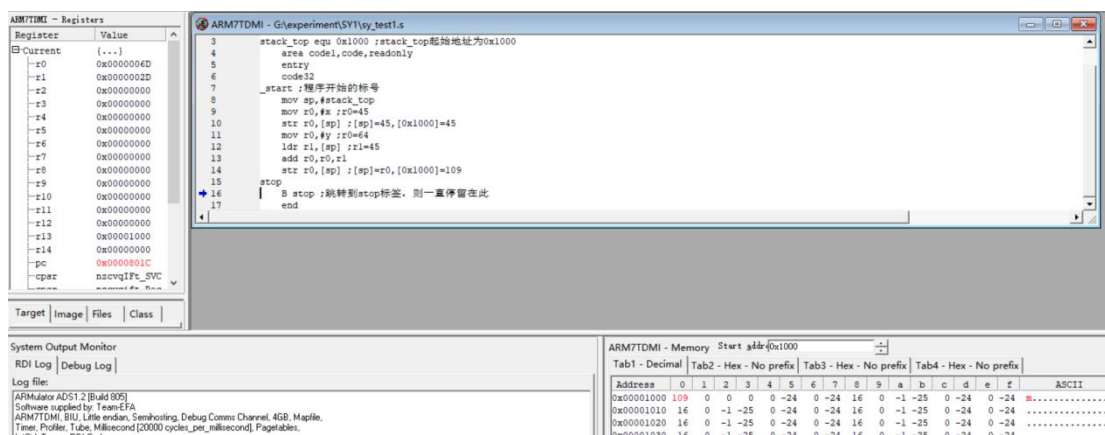


图 4 运行结束的寄存器及存储器值

3、实验练习 2 (ARM 指令寻址方式)

实验内容：验证 ARM 的寄存器和立即数寻址方式。程序的功能是：首先将立即数 15，送到寄存器 R0 中，接着再把立即数 8 送到寄存器 R1 中，然后使用寄存器寻址的方式将 R0 和 R1 中的数相减，结果送入 R0。由图可以看出 R0 存放 15-8=7。

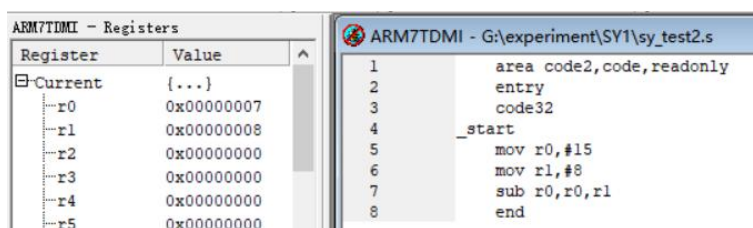


图 5 寄存器寻址

4、实验练习 3（验证 ARM 的寄存器偏移寻址方式）

实验内容：程序功能是：首先分别给寄存器 R0, R1, R2, R3 赋值为 2, 再将 R2 寄存器的值左移三位所得结果赋给 R0(R2 值不变), 最后将 R1 减去 R2 左移 R3(寄存器内的值)位的值, 并将结果赋给 R0。

R2 左移三位把值赋给 R0=0x00000010, 再 R0=R1-8R2=0xFFFFFFFF2

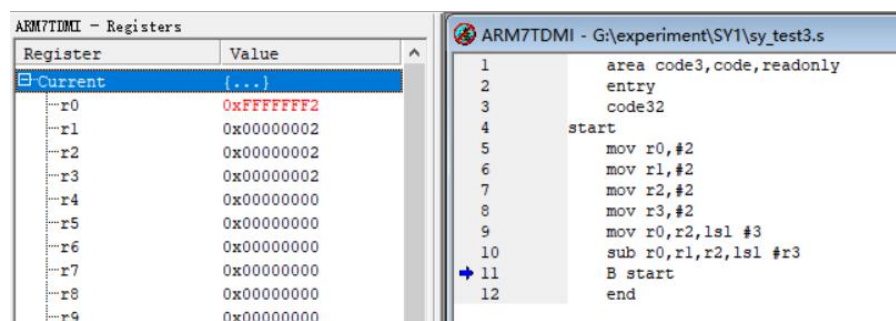


图 6 寄存器偏移寻址

5、实验练习 4（验证 ARM 的寄存器间接寻址方式）

程序功能是：先给 R0 寄存器用立即数寻址方式赋一个值, 然后将 R0 进行移位操作使它成为指向内存指定位置的地址, 然后将该地址的内容提取到 R1 寄存器中, 接着进行交换操作。

左移后 R0=R0*8=0x00000010, 再 LDR 将该地址的内容提取到 R1 寄存器中, 最后交换使用 swp 操作, 如图所示:

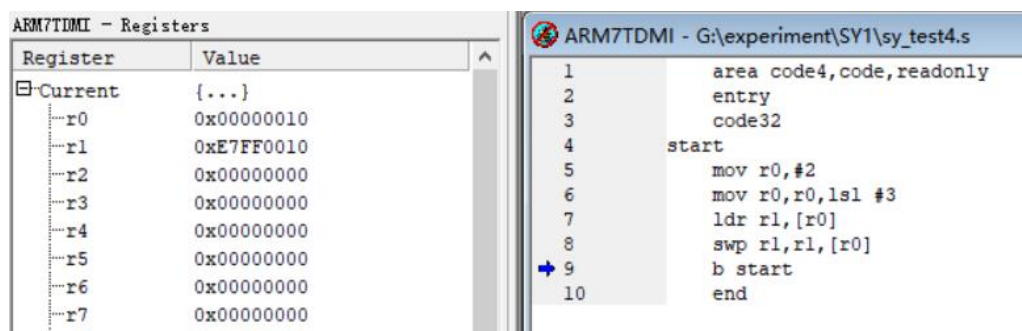


图 7 寄存器间接寻址

6、实验练习 5（验证 ARM 的堆栈寻址方式）

程序功能是：分别给 R0, R1, R2, R3, R4, LR 赋值为 1, 2, 3, 4, 5, 6, 堆栈栈顶指针赋值 0x20, 将 R0-R4, LR 通过满递减（或者 FA, EA, ED）压入堆栈，并从堆栈中将数据弹出至 R6, R7, R8, R9, R10, R11，观察堆栈指针、堆栈区变化以及寄存器内容变化。

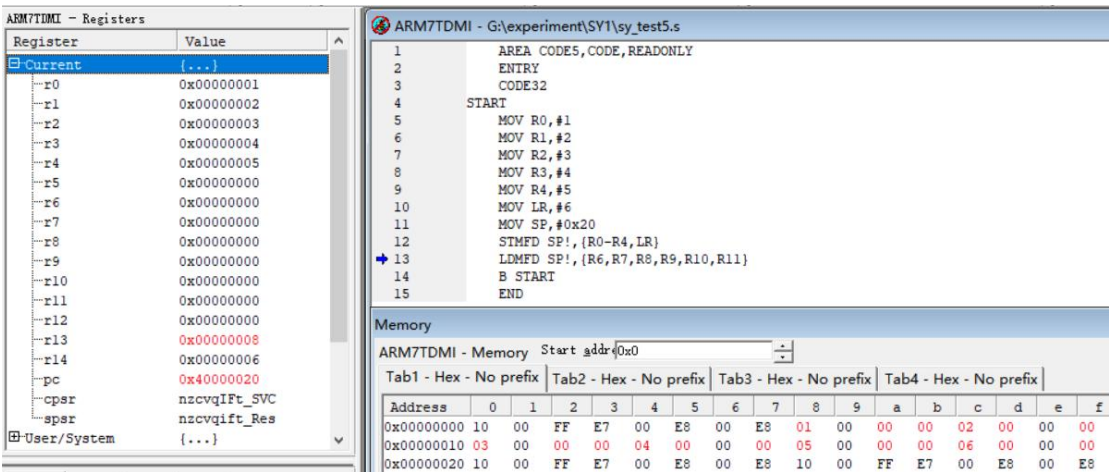


图 8 压入栈的存储器中的值

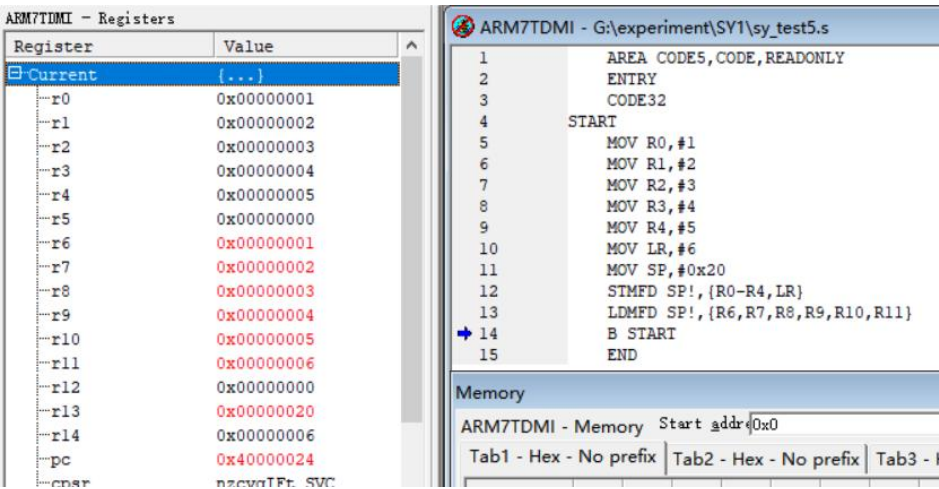


图 9 弹出栈的寄存器中的值

7、实验练习 6

实验内容：输入以下指令（1）并写出指令的寻址方式（2）单步调试指令（1）-（10），观察相关寄存器和存储器的变化，并把结果填写到表格中（表格中☆标注的地方）

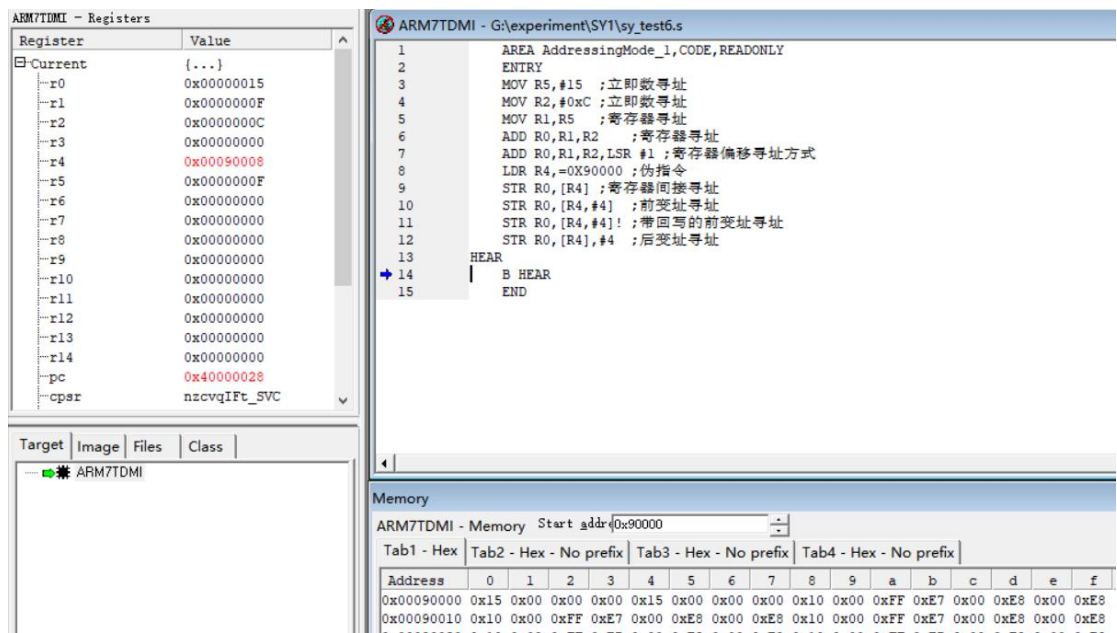


图 10 运行结果图

表 1 指令执行后的变化情况

序号	寻址方式	执行指令	指令执行后的变化情况											
			寄存器(值用16进制表示)						存储器(值用16进制表示)					
			R0	R1	R2	R3	R4	R5						
1	立即数寻址	MOV R5, #15						0x0000000F						
2	立即数寻址	MOV R2, #0xC			0x0000000C									
3	寄存器寻址	MOV R1, R5		0x0000000F										
4	寄存器寻址	ADD R0, R1, R2	0x0000001B											
5	寄存器偏移寻址	ADD R0, R1, R2, LSR #1	0x00000015	0x0000000F	0x0000000C									
6	伪指令	LDR R4, =0x90000					0x00090000							
7	寄存器间接寻址	STR R0, [R4]							0x90000	0x90001	0x90002	0x90003		
8	前变址寻址	STR R0, [R4, #4]							0x15	0x00	0x00	0x00		
9	带写回的前变址寻址	STR R0, [R4, #4]!							0x15	0x00	0x00	0x00		
10	后变址寻址	STR R0, [R4], #4							0x15	0x00	0x00	0x00		

四、实验心得

1、写出自己的收获

在完成这些实验的过程中，我深入了解了嵌入式微处理器系统的开发环境和相关配置。通过实验，我学会了创建嵌入式系统的工程，

添加文件，设置链接地址等基本操作。我也了解了 ARM 指令集中的不同寻址方式，包括立即数寻址、寄存器寻址、寄存器偏移寻址、寄存器间接寻址和堆栈寻址。这些寻址方式对于实现不同的功能非常重要。

2、实验过程中出现的问题

(1) 环境配置问题：在建立嵌入式系统的开发环境时，可能会遇到配置问题，例如选择适合的工程模板、正确设置链接地址等。

(2) 编写程序问题：在实验练习中编写程序时，由于缩进问题花费了些时间去调试代码。

(3) 调试问题：在观察数据变化或寄存器状态时，包括设置断点、单步运行代码、观察变量值等。